

Linear Regression with multiple variables

Multiple features

- Different names
- Number of dependent & independent variables
- Regression and nature of dependent variable
- Nature of the model assuming $y=f(x_1, x_2, \dots, x_n)$
- House Price Prediction Example (Next Slide)

Multiple features (variables).

Size (feet ²)	Price (\$1000)
$\rightarrow x$	$y \leftarrow$
2104	460
1416	232
1534	315
852	178
...	...

$$\underline{h_{\theta}(x) = \theta_0 + \theta_1 x}$$

Multiple features (variables).

Size (feet ²)	Number of bedrooms	Number of floors	Age of home (years)	Price (\$1000)
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
...	...	...	...	...

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

Multiple features (variables).

Size (feet ²)	Number of bedrooms	Number of floors	Age of home (years)	Price (\$1000)
x_1	x_2	x_3	x_4	y
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
...	...	...	...	...

$m = 47$

Notation:

- $\rightarrow n$ = number of features $n = 4$
- $\rightarrow x^{(i)}$ = input (features) of i^{th} training example.
- $\rightarrow x_j^{(i)}$ = value of feature j in i^{th} training example.

$$\underline{x^{(2)}} = \begin{bmatrix} 1416 \\ 3 \\ 2 \\ 40 \end{bmatrix}$$

$x_3^{(2)} = 2$

Hypothesis:

Previously: $h_{\theta}(x) = \theta_0 + \theta_1 x$

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \theta_4 x_4$$

e.g. $\underline{h_0(x)} = \underline{80} + \underline{0.1x_1} + \underline{0.01x_2} + 3x_3 - 2x_4$
↑ ↑ ↑
age

$$\rightarrow h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

For convenience of notation, define $x_0 = 1$. ($x_0^{(i)} = 1$)

$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^{n+1}$$

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \\ \theta_n \end{bmatrix} \in \mathbb{R}^{n+1}$$

$$h_{\theta}(x) = \theta_0 x_0 + \theta_1 x_1 + \dots + \theta_n x_n$$

$$= \theta^T x$$

$$[\theta_0 \ \theta_1 \ \dots \ \theta_n]$$

$$\theta^T$$

(n+1) x 1
matrix

$$\theta^T x$$

$$\begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_n \end{bmatrix}$$

x

Multivariate linear regression. $\leftarrow$

Linear Regression with
multiple variables

Gradient descent for
multiple variables

Hypothesis: $h_{\theta}(x) = \theta^T x = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$

Parameters: $\theta_0, \theta_1, \dots, \theta_n$

Cost function:

$$J(\theta_0, \theta_1, \dots, \theta_n) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Gradient descent:

Repeat {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \dots, \theta_n)$$

} (simultaneously update for every $j = 0, \dots, n$)

Hypothesis: $h_{\theta}(x) = \theta^T x = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$

Handwritten notes: $x_0 = 1$ (green arrow pointing to x_0)

Parameters: $\theta_0, \theta_1, \dots, \theta_n$

Handwritten notes: θ (blue circle), $n+1$ -dimensional vector (blue text)

Cost function:

$$J(\theta_0, \theta_1, \dots, \theta_n) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Handwritten notes: $J(\theta)$ (blue text below the parameters in the cost function)

Gradient descent:

Repeat {

$\rightarrow \theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \dots, \theta_n)$

Handwritten notes: $J(\theta)$ (blue text below the cost function), (simultaneously update for every $j = 0, \dots, n$) (black text)

Gradient Descent

Previously ($n=1$):

Repeat {

→ $\theta_0 := \theta_0 - \alpha \underbrace{\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})}_{\frac{\partial}{\partial \theta_0} J(\theta)}$

→ $\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \underbrace{x_1^{(i)}}_{\text{simultaneously update } \theta_0, \theta_1}$

}

New algorithm ($n \geq 1$):

Repeat {

→ $\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$
 (simultaneously update θ_j for $j = 0, \dots, n$)

}

→ $\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \underbrace{x_0^{(i)}}_{x_0^{(i)} = 1}$

→ $\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_1^{(i)}$

→ $\theta_2 := \theta_2 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_2^{(i)}$

...

Linear Regression with multiple variables

Gradient descent in
practice I: Feature Scaling

Feature Scaling

Get every feature into approximately a $-1 \leq x_i \leq 1$ range.

$$x_0 = 1$$

$$0 \leq x_1 \leq 3 \quad \checkmark$$

$$-2 \leq x_2 \leq 0.5 \quad \checkmark$$

$$-100 \leq x_3 \leq 100 \quad \times$$

$$-0.0001 \leq x_4 \leq 0.0001 \quad \times$$

$$-3 \text{ to } 3 \quad \checkmark$$

$$-\frac{1}{3} \text{ to } \frac{1}{3} \quad \checkmark$$

Min-max normalization performs a linear transformation on the original data. Suppose that $\min_A$ and $\max_A$ are the minimum and maximum values of an attribute, A . Min-max normalization maps a value, v , of A to v' in the range $[\text{new_min}_A, \text{new_max}_A]$ by computing

$$v' = \frac{v - \min_A}{\max_A - \min_A} (\text{new_max}_A - \text{new_min}_A) + \text{new_min}_A. \quad (2.11)$$

Min-max normalization preserves the relationships among the original data values. It will encounter an “out-of-bounds” error if a future input case for normalization falls outside of the original data range for A .

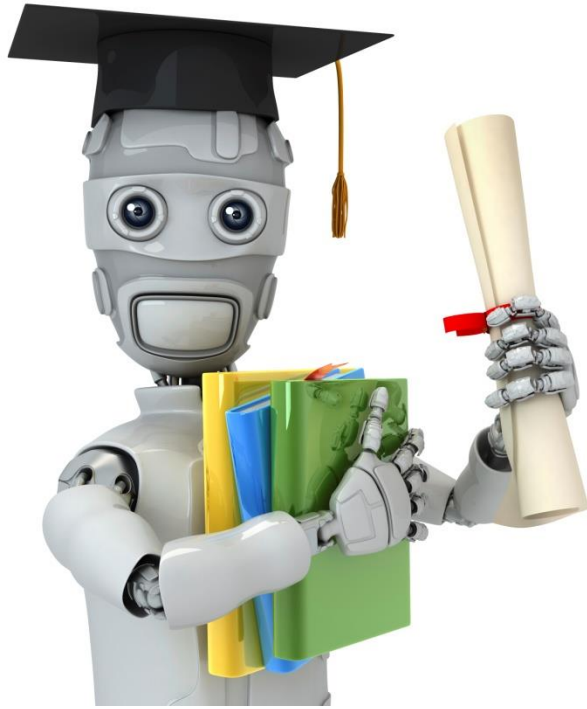
Min-max normalization. Suppose that the minimum and maximum values for the attribute *income* are \$12,000 and \$98,000, respectively. We would like to map *income* to the range $[0.0, 1.0]$. By min-max normalization, a value of \$73,600 for *income* is transformed to $\frac{73,600 - 12,000}{98,000 - 12,000} (1.0 - 0) + 0 = 0.716$. ■

In **z-score normalization** (or *zero-mean normalization*), the values for an attribute, A , are normalized based on the mean and standard deviation of A . A value, v , of A is normalized to v' by computing

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2 \qquad v' = \frac{v - \bar{A}}{\sigma_A}, \qquad (2.12)$$

where $\bar{A}$ and σ_A are the mean and standard deviation, respectively, of attribute A . This method of normalization is useful when the actual minimum and maximum of attribute A are unknown, or when there are outliers that dominate the min-max normalization.

z-score normalization Suppose that the mean and standard deviation of the values for the attribute *income* are \$54,000 and \$16,000, respectively. With z-score normalization, a value of \$73,600 for *income* is transformed to $\frac{73,600 - 54,000}{16,000} = 1.225$. ■



Machine Learning

Linear Regression with multiple variables

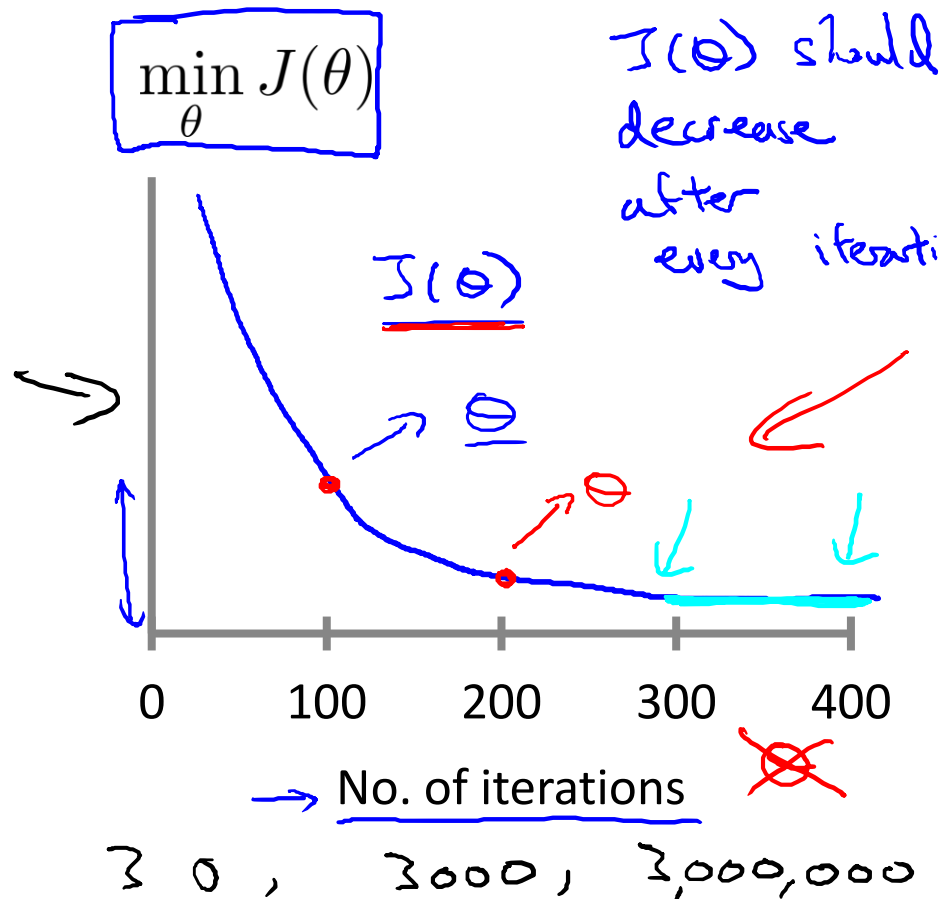
Gradient descent in practice II: Learning rate

Gradient descent

$$\rightarrow \theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

- “Debugging”: How to make sure gradient descent is working correctly.
- How to choose learning rate α .

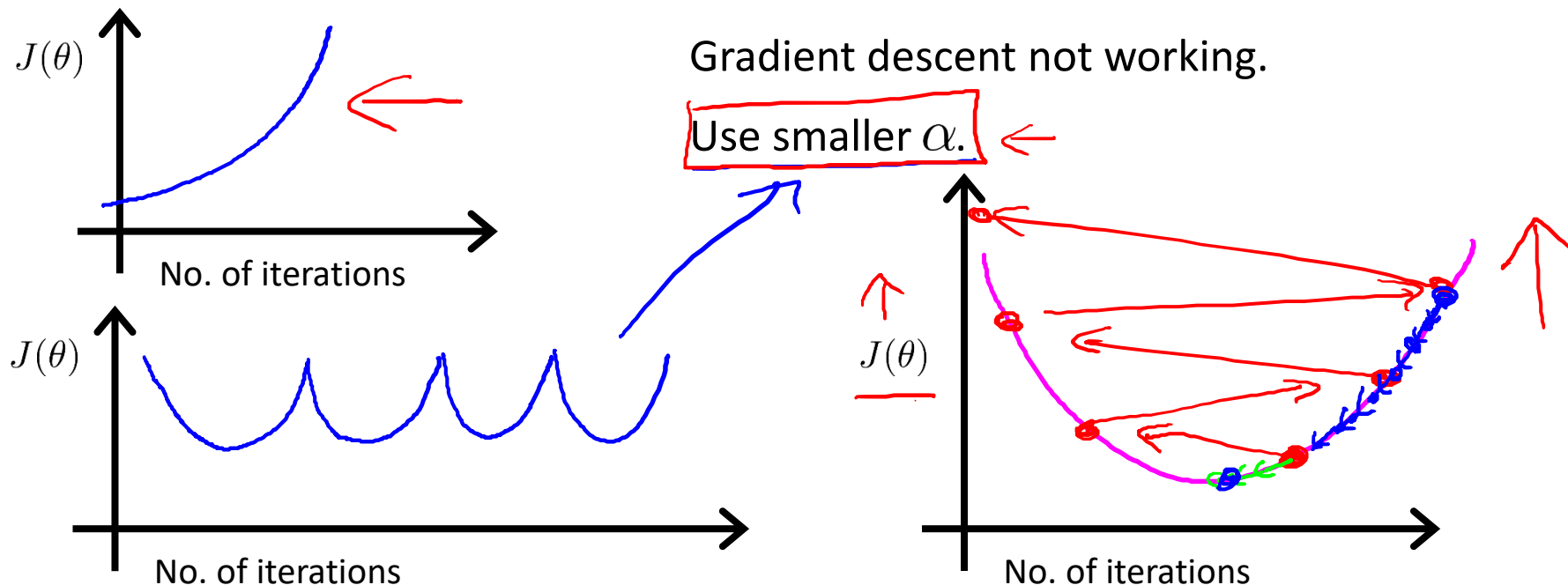
Making sure gradient descent is working correctly.



→ Example automatic convergence test:

→ Declare convergence if $J(\theta)$ decreases by less than 10^{-3} in one iteration.

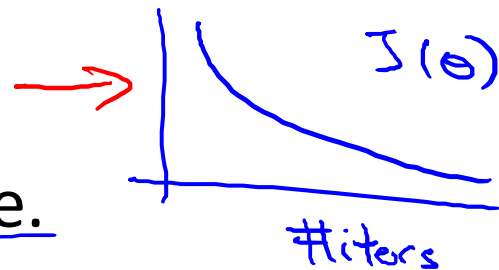
Making sure gradient descent is working correctly.



- For sufficiently small α , $J(\theta)$ should decrease on every iteration. ←
- But if α is too small, gradient descent can be slow to converge.

Summary:

- If α is too small: slow convergence.
- If α is too large: $J(\theta)$ may not decrease on every iteration; may not converge.

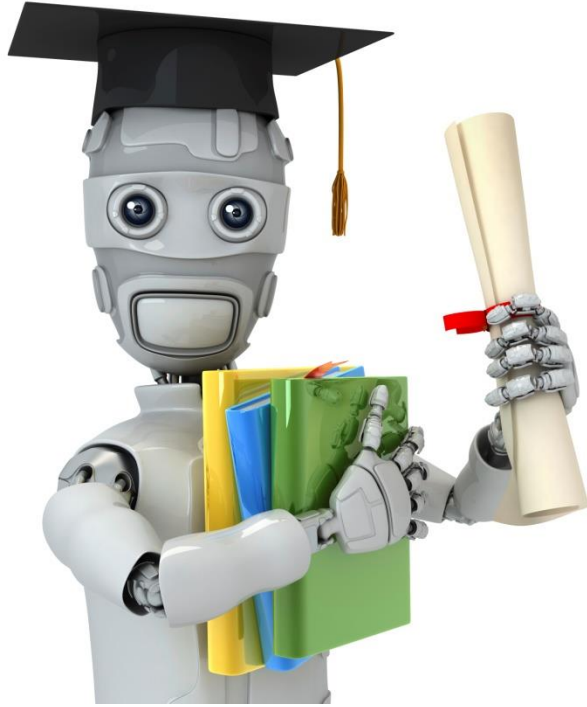


(Slow converge also possible.)

To choose α , try

$\dots, \underline{0.001}, \underline{0.003}, \underline{0.01}, \underline{0.03}, \underline{0.1}, \underline{0.3}, \underline{1}, \dots$

Arrows indicate a sequence of values: $0.001 \rightarrow 0.003$ (labeled $3\times$), $0.003 \rightarrow 0.01$ (labeled $\approx 3\times$), $0.01 \rightarrow 0.03$ (labeled $3\times$), $0.03 \rightarrow 0.1$ (labeled $\approx 3\times$), and $0.1 \rightarrow 0.3$ (labeled $3\times$). There are also arrows pointing up to 0.001 and 1 .

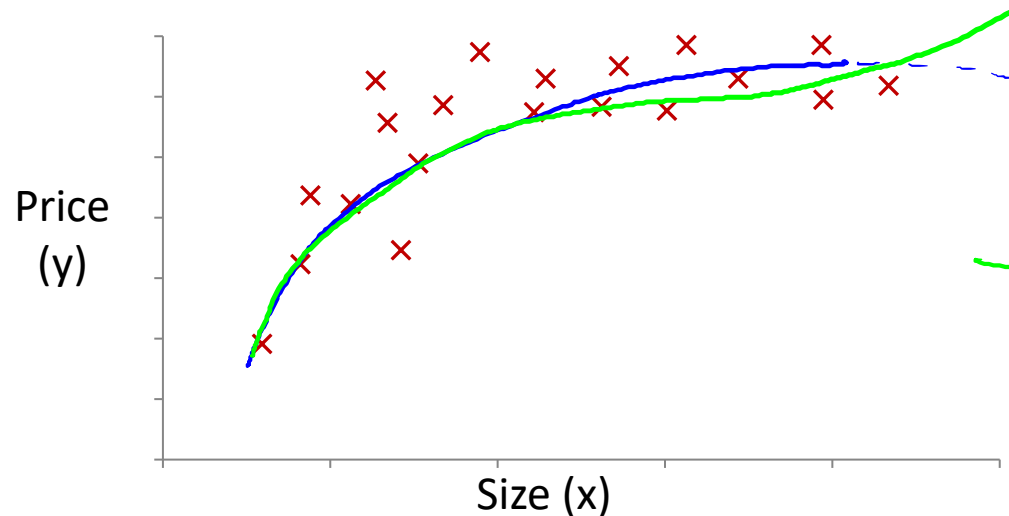


Machine Learning

Linear Regression with multiple variables

Features and polynomial regression

Polynomial regression



$$\theta_0 + \theta_1 x + \theta_2 x^2$$

$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3$$

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$$

$$= \theta_0 + \theta_1(\text{size}) + \theta_2(\text{size})^2 + \theta_3(\text{size})^3$$

$$\rightarrow x_1 = (\text{size})$$

$$\rightarrow x_2 = (\text{size})^2$$

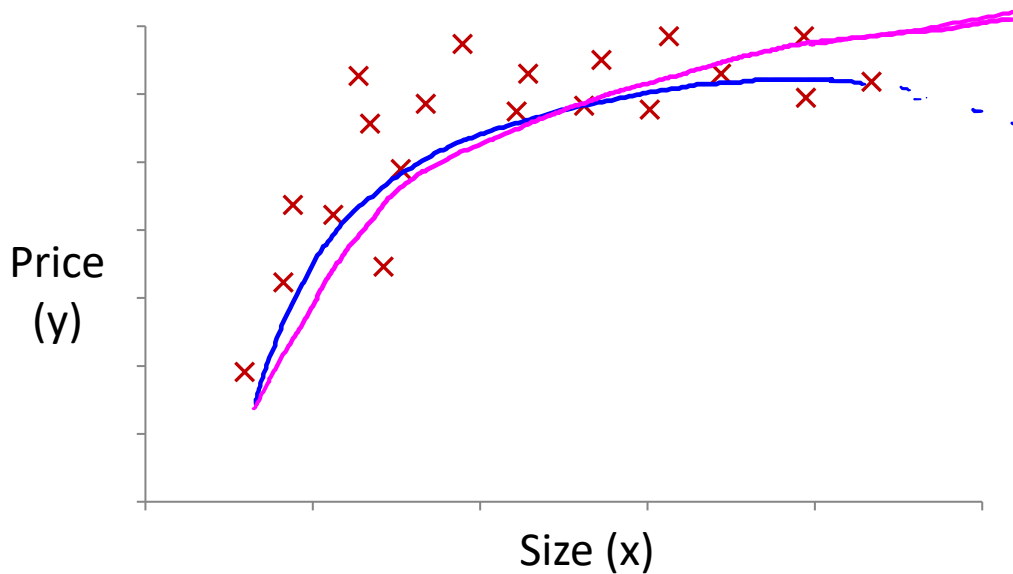
$$\rightarrow x_3 = (\text{size})^3$$

$$\text{Size: } 1 - 1000$$

$$\text{Size}^2: 1 - 1,000,000$$

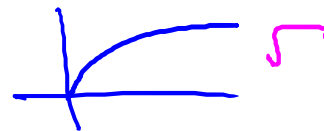
$$\text{Size}^3: 1 - 10^9$$

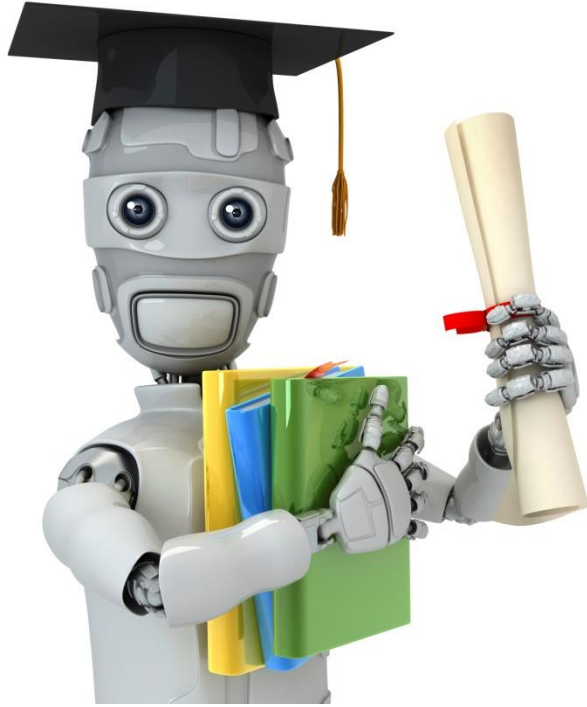
Choice of features



→ $h_{\theta}(x) = \theta_0 + \theta_1(\text{size}) + \theta_2(\text{size})^2$

→ $h_{\theta}(x) = \theta_0 + \theta_1(\text{size}) + \theta_2\sqrt{(\text{size})}$



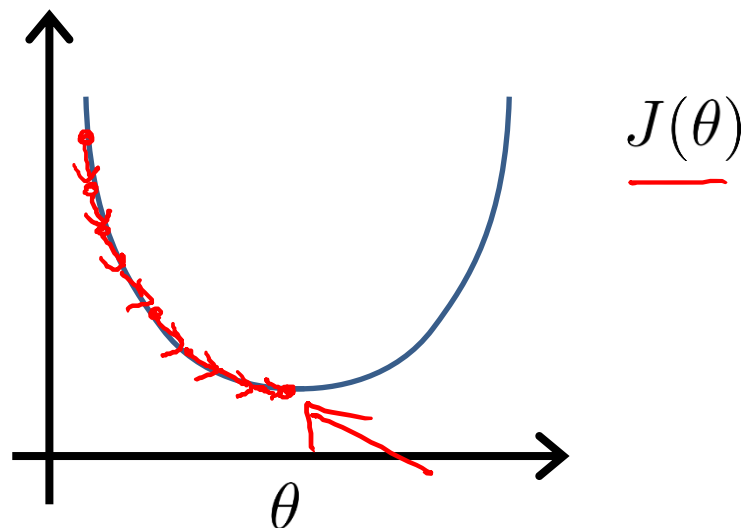


Machine Learning

Linear Regression with multiple variables

Normal equation

Gradient Descent



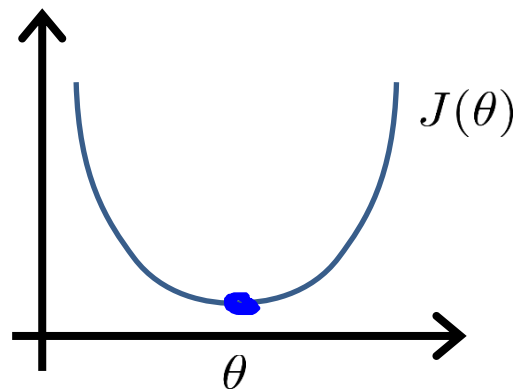
Normal equation: Method to solve for θ
analytically.

Intuition: If 1D ($\theta \in \mathbb{R}$)

$\rightarrow$ $J(\theta) = a\theta^2 + b\theta + c$

$\frac{d}{d\theta} J(\theta) = \dots \stackrel{\text{set}}{=} 0$

Solve for θ



$\theta \in \mathbb{R}^{n+1}$ $J(\theta_0, \theta_1, \dots, \theta_m) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$

$\frac{\partial}{\partial \theta_j} J(\theta) = \dots \stackrel{\text{set}}{=} 0$ (for every j)

Solve for $\theta_0, \theta_1, \dots, \theta_n$

Examples: $m = 4$.

	Size (feet ²)	Number of bedrooms	Number of floors	Age of home (years)	Price (\$1000)
x_0	x_1	x_2	x_3	x_4	y
1	2104	5	1	45	460
1	1416	3	2	40	232
1	1534	3	2	30	315
1	852	2	1	36	178

$X = \begin{bmatrix} 1 & 2104 & 5 & 1 & 45 \\ 1 & 1416 & 3 & 2 & 40 \\ 1 & 1534 & 3 & 2 & 30 \\ 1 & 852 & 2 & 1 & 36 \end{bmatrix}$
 $m \times (n+1)$

$y = \begin{bmatrix} 460 \\ 232 \\ 315 \\ 178 \end{bmatrix}$
 m -dimensional vector

$\theta = (X^T X)^{-1} X^T y$

$$\theta = (X^T X)^{-1} X^T y$$

Show the derivation for this equation on Board.

$(X^T X)^{-1}$ is inverse of matrix $X^T X$.

Set $A = X^T X$

$$(X^T X)^{-1} = A^{-1}$$

Octave: `pinv(X' * X) * X' * y`

$$\text{pinv}(X^T * X) * X^T * y$$

$$\theta = (X^T X)^{-1} X^T y$$

$\min J(\theta)$

X'

X^T

~~Feature Scaling~~

$$0 \leq x_1 \leq 1$$

$$0 \leq x_2 \leq 1000$$

$$0 \leq x_3 \leq 10^{-5}$$



m training examples, n features.

Gradient Descent

- • Need to choose α .
- • Needs many iterations.
- Works well even when n is large.



$$\underline{n = 10^6}$$

← -

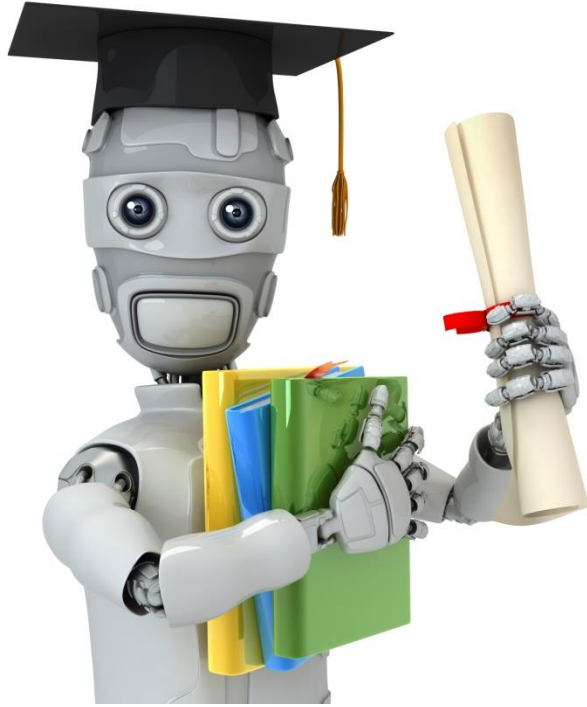
Normal Equation

- • No need to choose α .
- • Don't need to iterate.
- Need to compute
- • $(X^T X)^{-1}$ $\underline{n \times n}$ $\underline{O(n^3)}$
- Slow if n is very large.

$$n = 100$$

$$n = 1000$$

$$- - - \underline{n = 10000}$$



Machine Learning

Linear Regression with multiple variables

Normal equation
and non-invertibility
(optional)

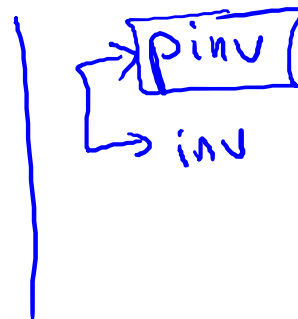
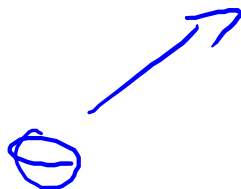
Normal equation

$$\theta = \underline{(X^T X)^{-1} X^T y}$$

$$\underline{X^T X}$$

- What if $\boxed{X^T X}$ is non-invertible? (singular/
degenerate)

- Octave: `pinv(X' * X) * X' * y`



What if $X^T X$ is non-invertible?

- Redundant features (linearly dependent).

E.g. $\begin{cases} \underline{x_1} = \text{size in feet}^2 \\ \underline{x_2} = \text{size in m}^2 \\ \underline{x_1 = (3.28)^2 x_2} \end{cases}$

$1\text{m} = 3.28 \text{ feet}$

$\rightarrow m = 10 \leftarrow$

$\rightarrow n = 100 \leftarrow$

$\odot \in \mathbb{R}^{101}$

- Too many features (e.g. $m \leq n$).

- Delete some features, or use regularization.

$\downarrow$ later

$$y = w_0 + w_1 x.$$

$$w_0 = \bar{y} - w_1 \bar{x}$$

$$w_1 = \frac{\sum_{i=1}^{|D|} (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{|D|} (x_i - \bar{x})^2}$$

Salary data.

x years experience	y salary (in \$1000s)
----------------------	-------------------------

3	30
8	57
9	64
13	72
3	36
6	43
11	59
21	90
1	20
16	83

Given the above data, we compute $\bar{x} = 9.1$ and $\bar{y} = 55.4$. Substituting these values into Equations (6.50) and (6.51), we get

$$w_1 = \frac{(3-9.1)(30-55.4) + (8-9.1)(57-55.4) + \cdots + (16-9.1)(83-55.4)}{(3-9.1)^2 + (8-9.1)^2 + \cdots + (16-9.1)^2} = 3.5$$

$$w_0 = 55.4 - (3.5)(9.1) = 23.6$$

Thus, the equation of the least squares line is estimated by $y = 23.6 + 3.5x$. Using this equation, we can predict that the salary of a college graduate with, say, 10 years of experience is \$58,600.

Error Measures

Absolute error : $|y_i - y'_i|$

Squared error : $(y_i - y'_i)^2$

Mean absolute error : $\frac{\sum_{i=1}^d |y_i - y'_i|}{d}$

Mean Absolute Percentage Error: $\frac{1}{d} \sum_{i=1}^d \left| \frac{y_i - y'_i}{y_i} \right|$

Root Mean Square Error: $\sqrt{\frac{\sum_{i=1}^d (y_i - y'_i)^2}{d}}$

$$NRMSE = \frac{RMSE}{y_{max} - y_{min}}$$

Evaluating Predictor

Training + Test Set (Holdout Set)

Cross Validation

Training + Validation + Test Set

Training + Test Set and Cross Validation of Training Set for Model Selection

References

- Andrew Ng's slides on Multiple Linear Regression from his Machine Learning Course on Coursera.
- Data Mining Book from Han and Kamber

Disclaimer

- Content of this presentation is not original and it has been prepared from various sources for teaching purpose.